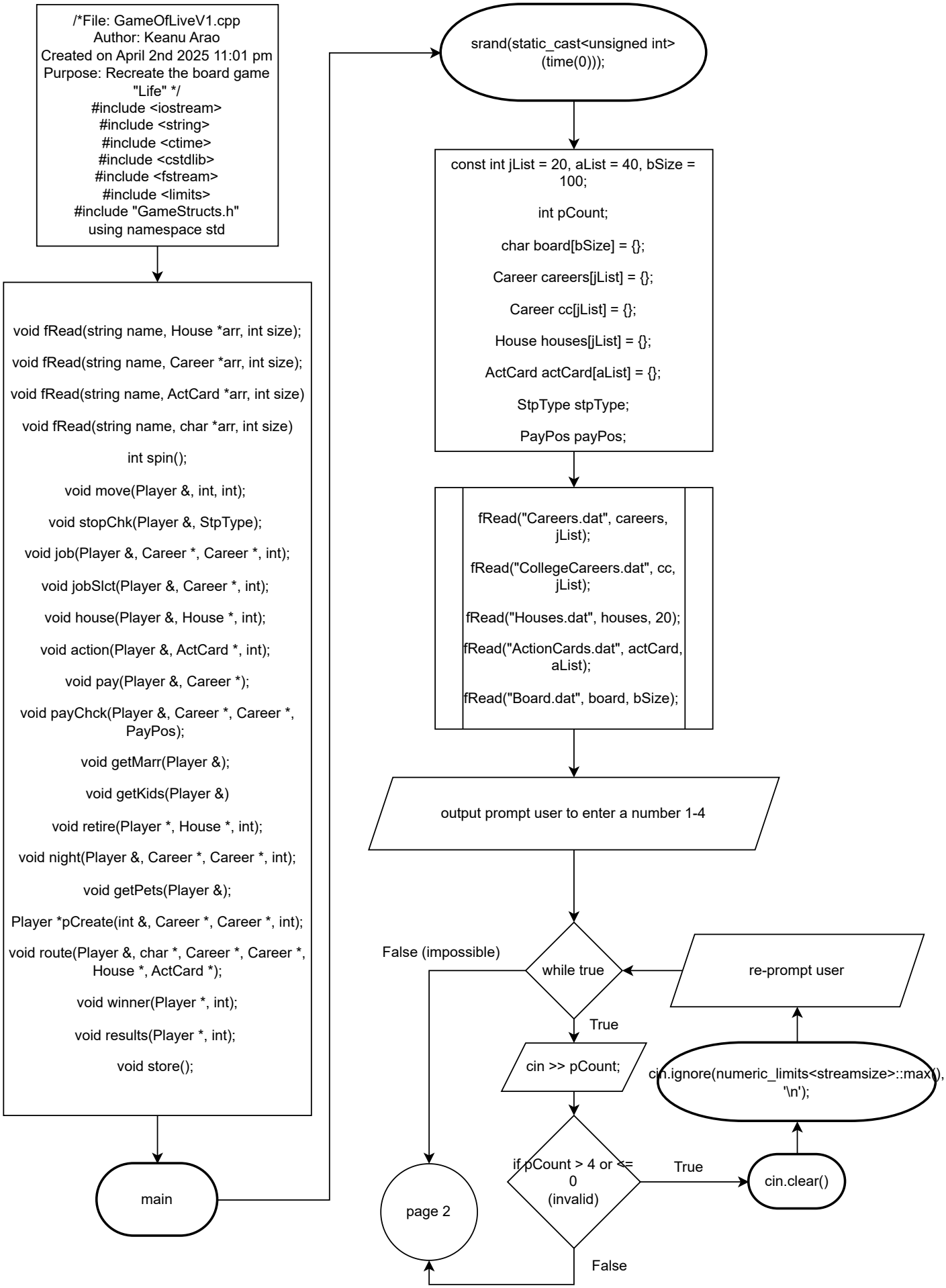
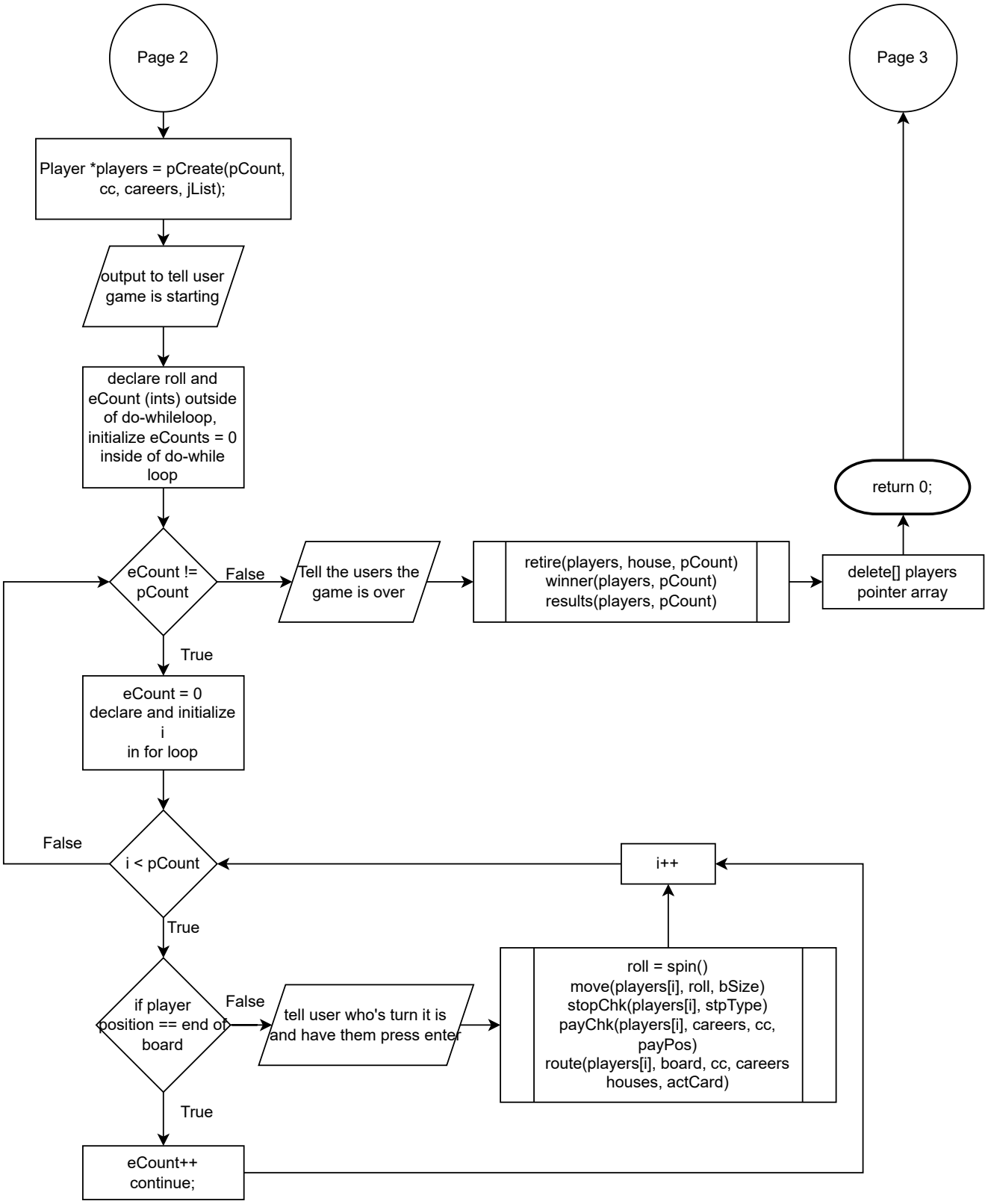
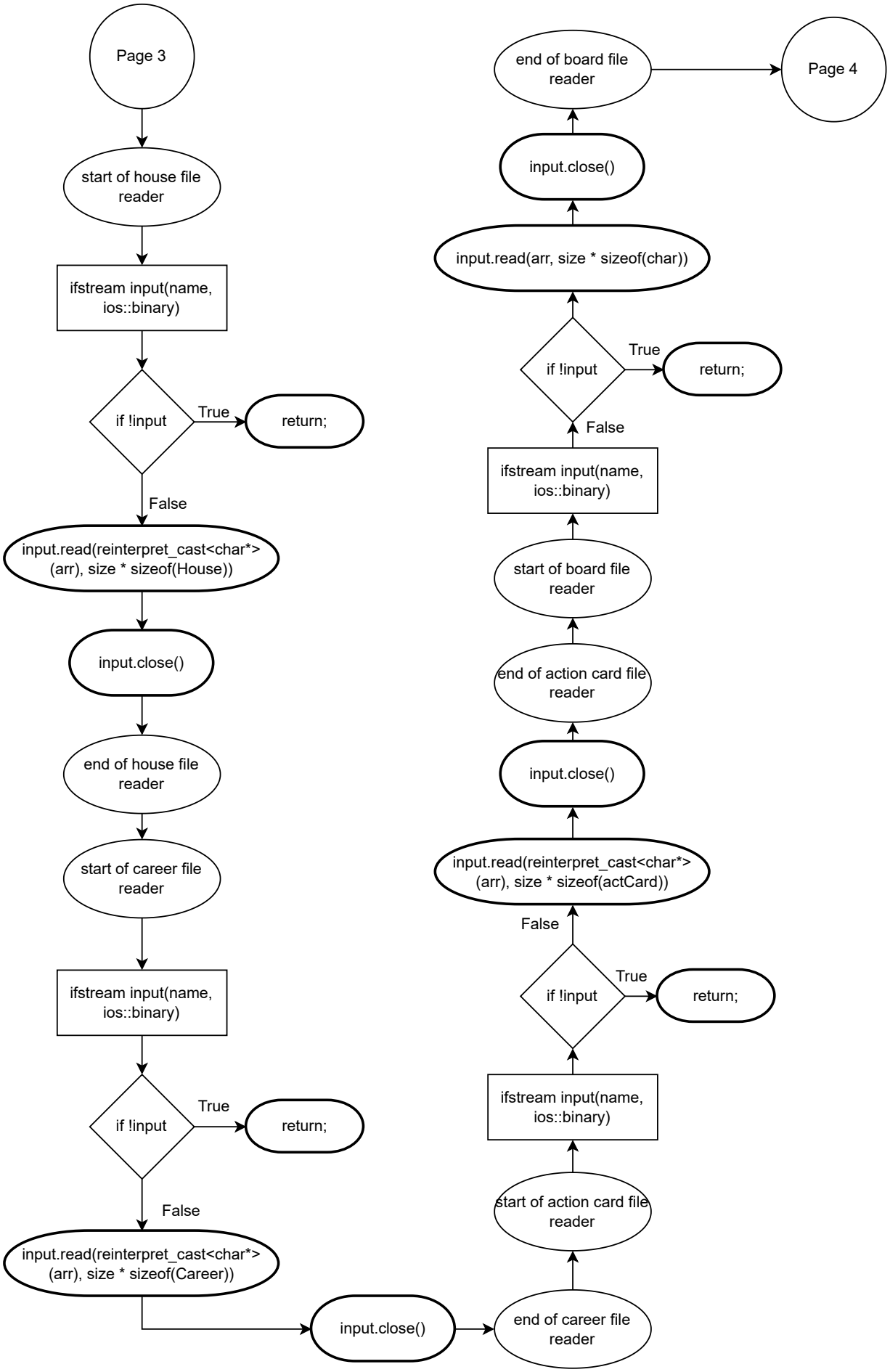
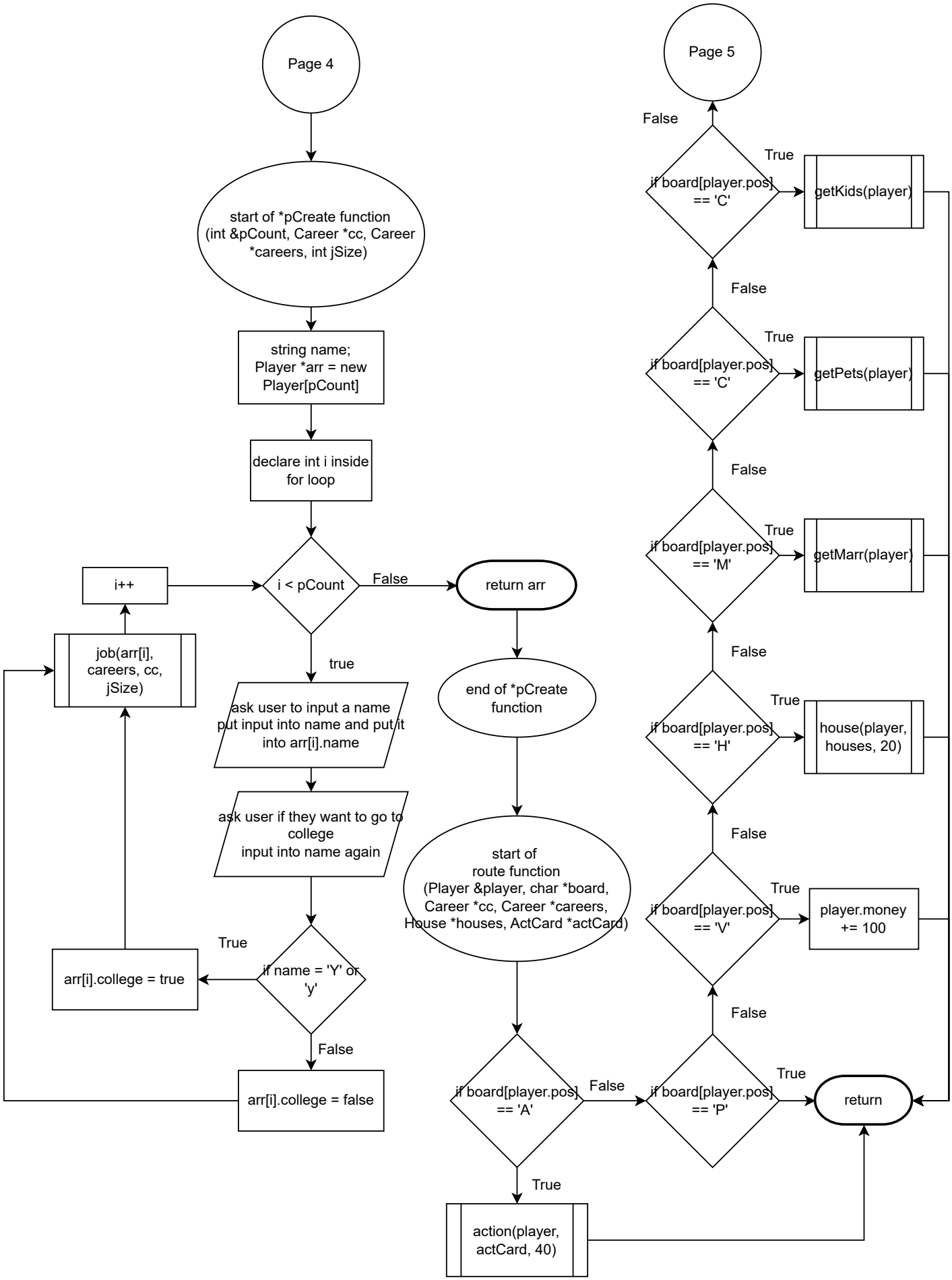
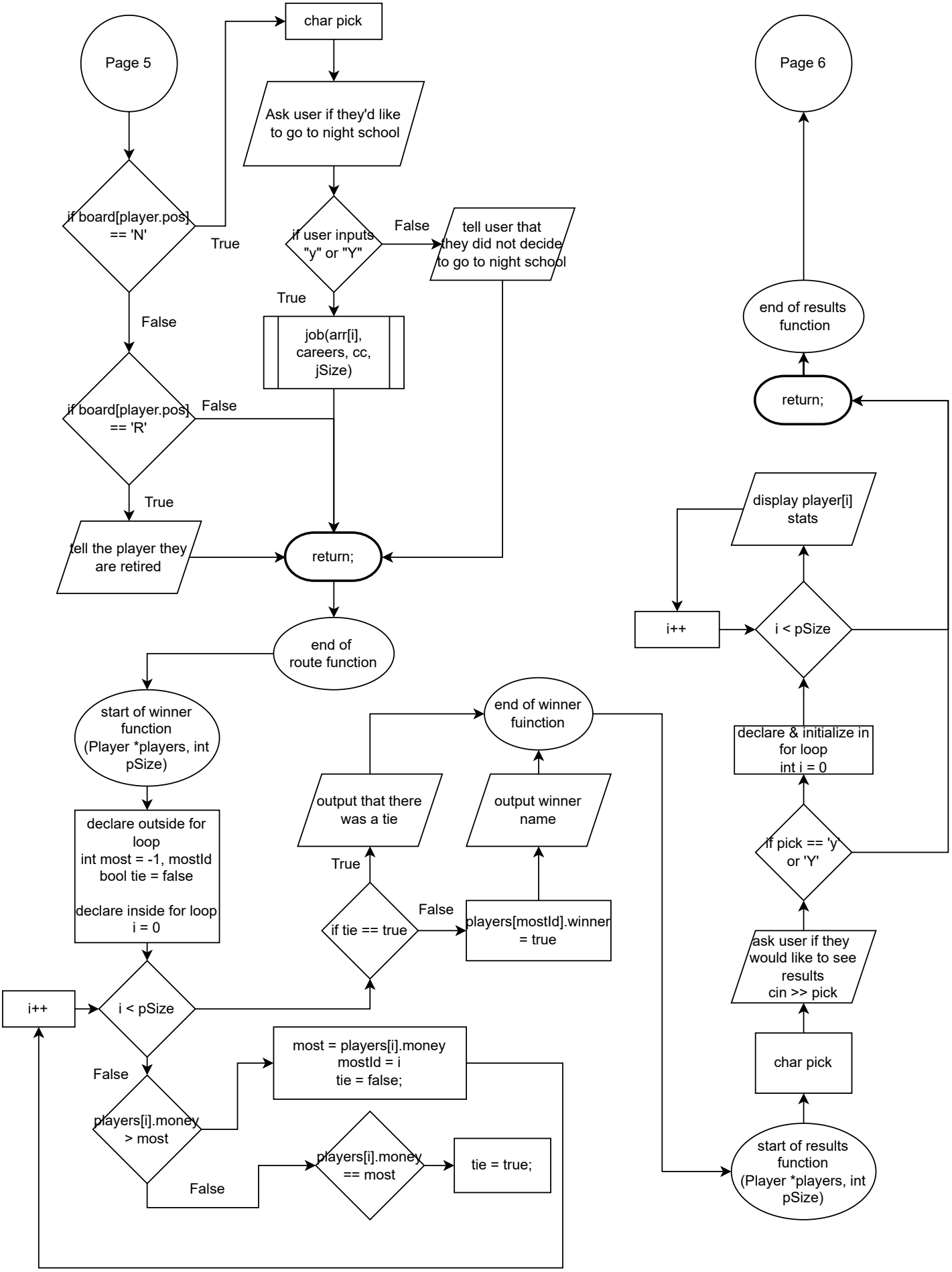
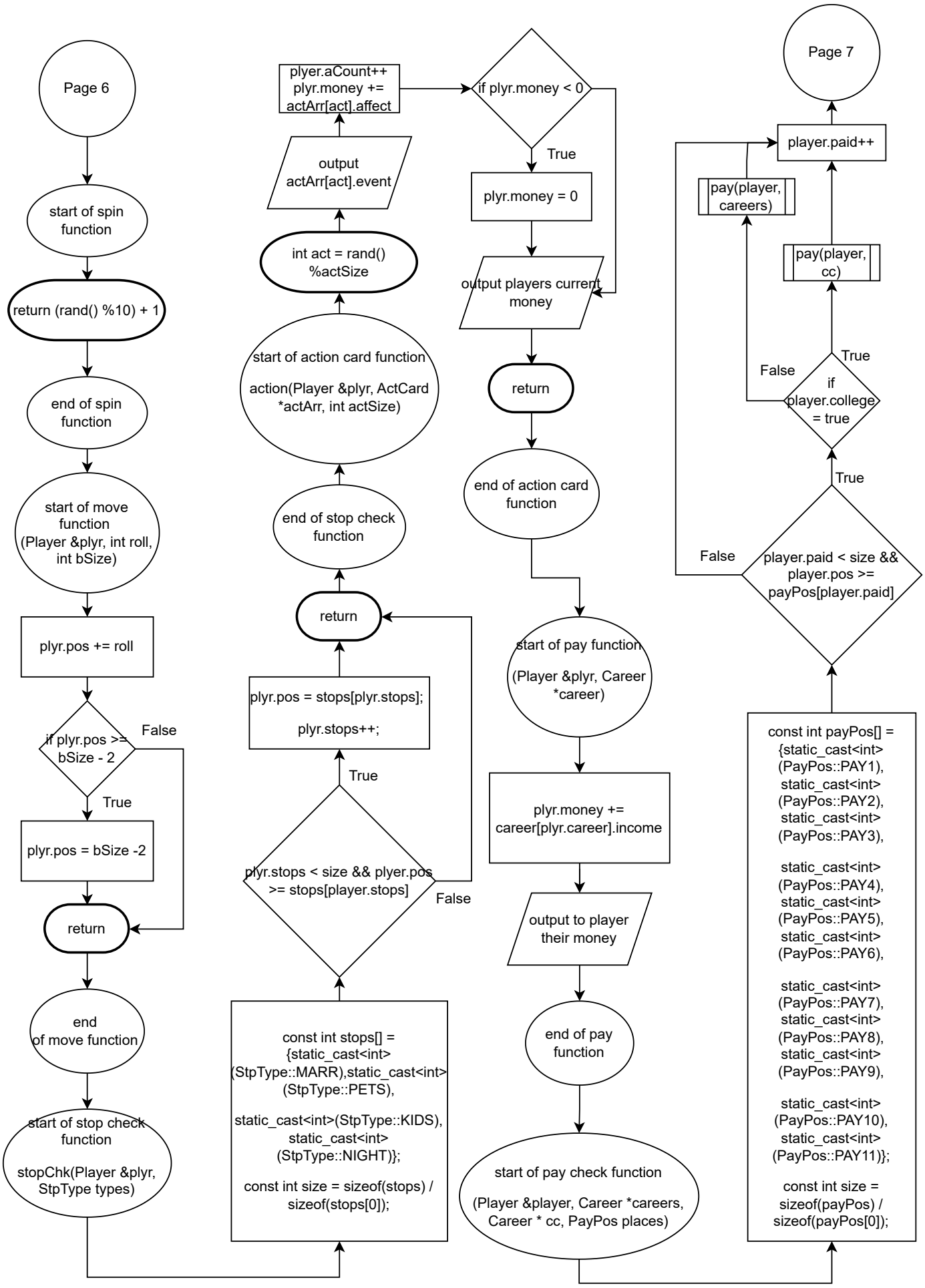

/*File: GameOfLiveV1.cpp
Author: Keanu Arao
Created on April 2nd 2025 11:01 pm
Purpose: Recreate the board game "Life" */
#include <iostream>
#include <string>
#include <ctime>
#include <cstdlib>
#include <fstream>
#include <limits>
#include "GameStructs.h"
using namespace std
void fRead(string name, House *arr, int size);
void fRead(string name, Career *arr, int size);
void fRead(string name, ActCard *arr, int size)
void fRead(string name, char *arr, int size)
int spin();
void move(Player &, int, int);
void stopChk(Player &, StpType);
void job(Player &, Career *, Career *, int);
void jobSlct(Player &, Career *, int);
void house(Player &, House *, int);
void action(Player &, ActCard *, int);
void pay(Player &, Career *);
void payChck(Player &, Career *, Career *, PayPos);
void getMarr(Player &);
void getKids(Player &)
void retire(Player *, House *, int);
void night(Player &, Career *, Career *, int);
void getPets(Player &);
Player *pCreate(int &, Career *, Career *, int);
void route(Player &, char *, Career *, Career *, House *, ActCard *);
void winner(Player *, int);
void results(Player *, int);
void store();
main
srand(static_cast<unsigned int>(time(0)));
const int jList = 20, aList = 40, bSize = 100;
int pCount;
char board[bSize] = {};
Career careers[jList] = {};
Career cc[jList] = {};
House houses[jList] = {};
ActCard actCard[aList] = {};
StpType stpType;
PayPos payPos;
fRead("Careers.dat", careers, jList);
fRead("CollegeCareers.dat", cc, jList);
fRead("Houses.dat", houses, 20);
fRead("ActionCards.dat", actCard, aList);
fRead("Board.dat", board, bSize);
output prompt user to enter a number 1-4
while true
False (impossible)
re-prompt user
True
cin >> pCount;
cin.ignore(numeric_limits<streamsize>::max(), '\n');
if pCount > 4 or <= 0 (invalid)
True
cin.clear()
page 2
False

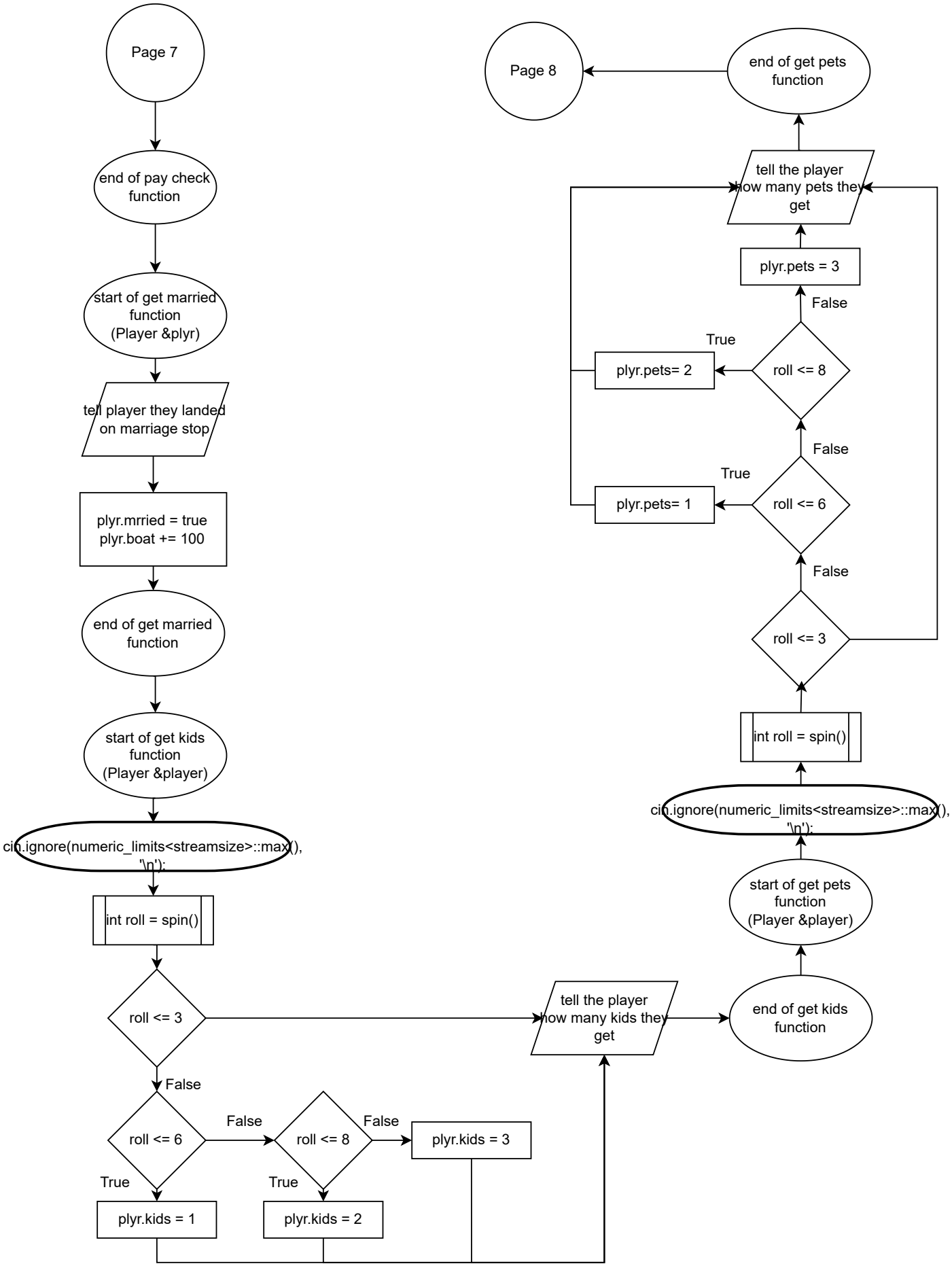

Page 2
Player *players = pCreate(pCount, cc, careers, jList);
output to tell user game is starting
declare roll and eCount (ints) outside of do-whileloop, initialize eCounts = 0 inside of do-while loop
eCount != pCount
False
Tell the users the game is over
retire(players, house, pCount)
winner(players, pCount)
results(players, pCount)
delete[] players pointer array
return 0;
Page 3
True
eCount = 0
declare and initialize i
in for loop
i < pCount
False
True
i++
if player position == end of board
False
tell user who's turn it is and have them press enter
roll = spin()
move(players[i], roll, bSize)
stopChk(players[i], stpType)
payChk(players[i], careers, cc, payPos)
route(players[i], board, cc, careers houses, actCard)
True
eCount++
continue;

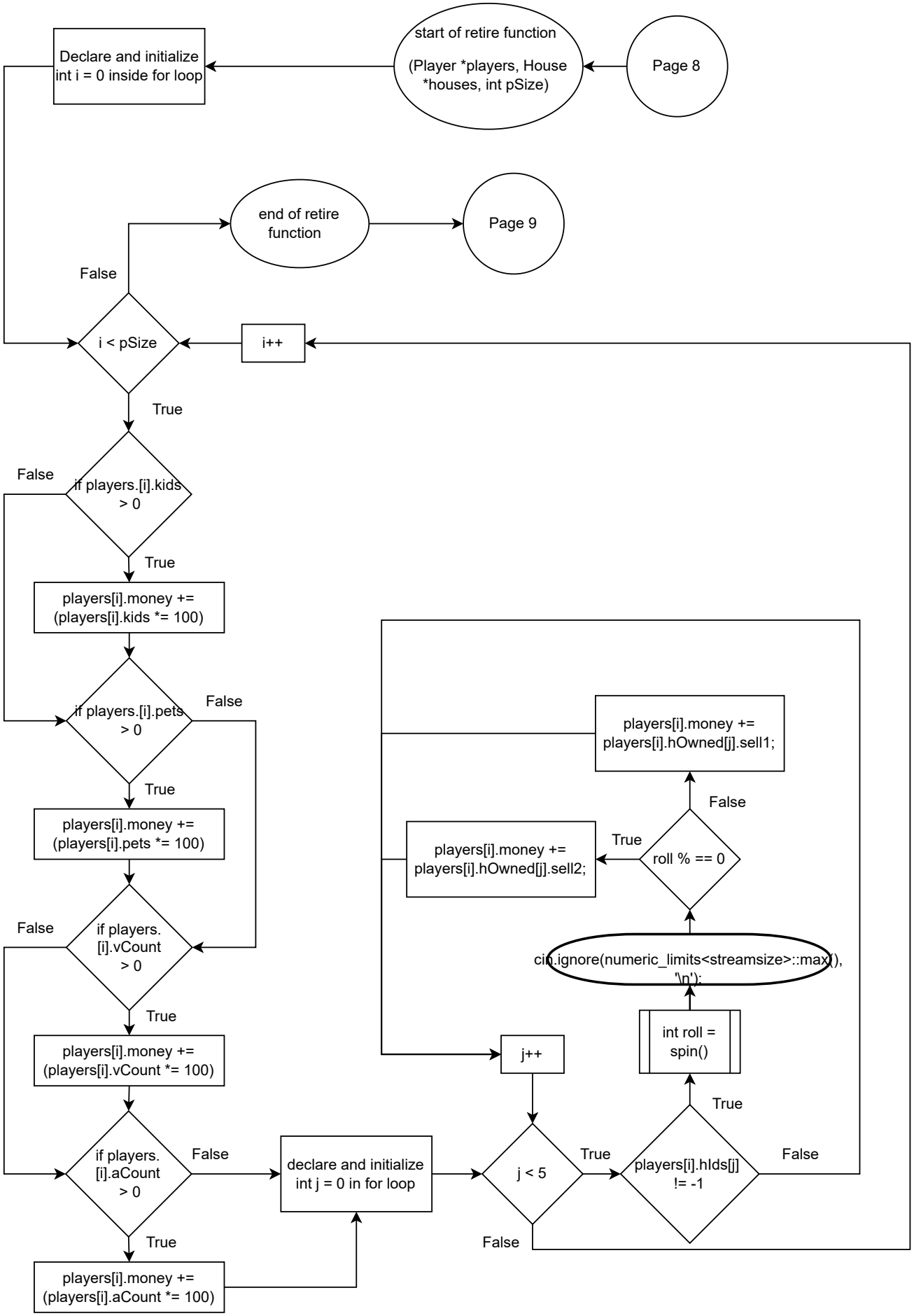

Page 3
start of house file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(House))
input.close()
end of house file reader
start of career file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(Career))
input.close()
end of career file reader
start of action card file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(reinterpret_cast<char*> (arr), size * sizeof(actCard))
input.close()
end of action card file reader
start of board file reader
ifstream input(name, ios::binary)
if !input
True
return;
False
input.read(arr, size * sizeof(char))
input.close()
end of board file reader
Page 4

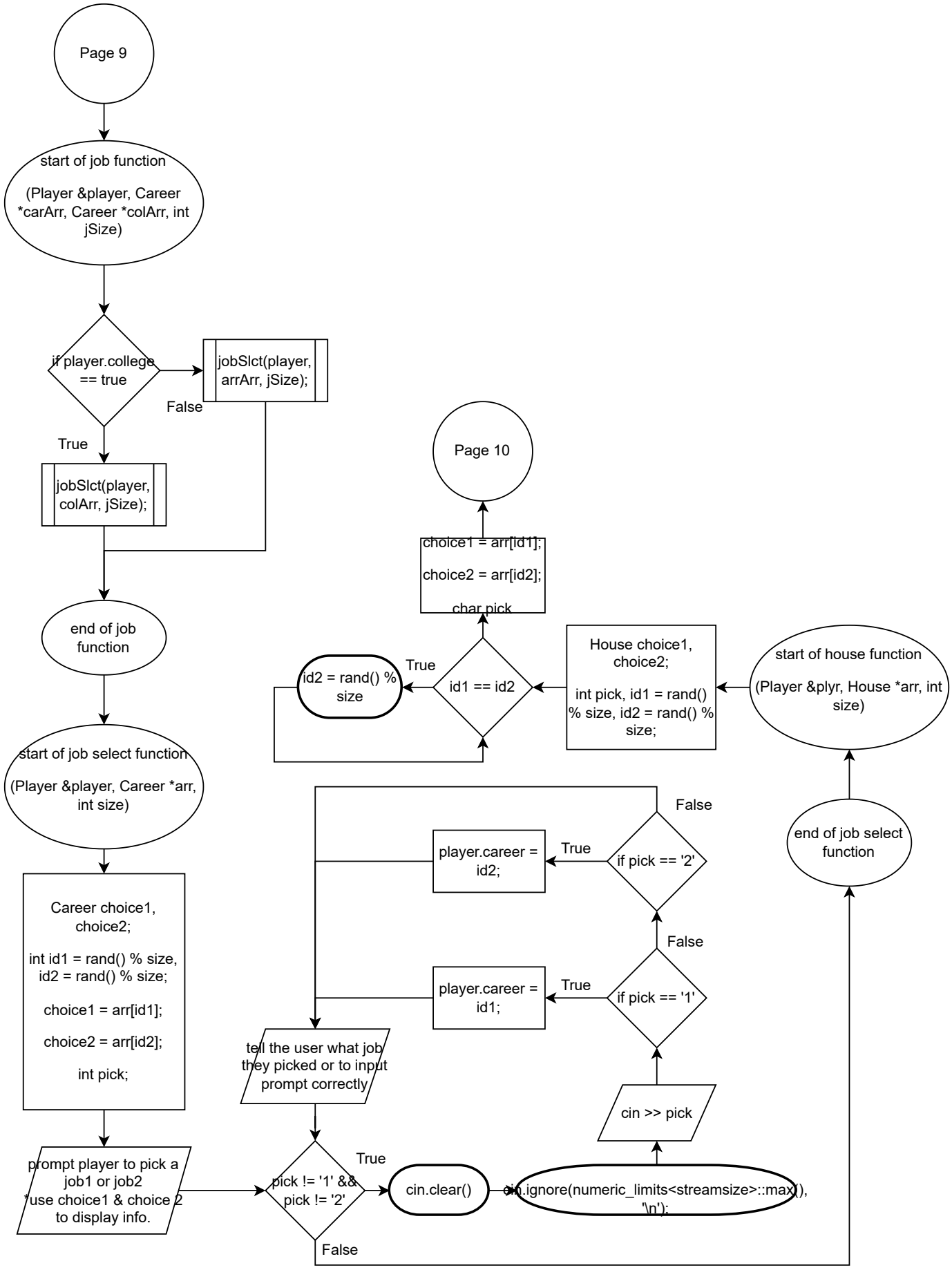

Page 4
start of *pCreate function (int &pCount, Career *cc, Career *careers, int jSize)
string name; Player *arr = new Player[pCount]
declare int i inside for loop
i < pCount
False
return arr
true
ask user to input a name put input into name and put it into arr[i].name
ask user if they want to go to college input into name again
if name = 'Y' or 'y'
True
arr[i].college = true
False
arr[i].college = false
job(arr[i], careers, cc, jSize)
i++
end of *pCreate function
start of route function (Player &player, char *board, Career *cc, Career *careers, House *houses, ActCard *actCard)
if board[player.pos] == 'A'
False
True
action(player, actCard, 40)
if board[player.pos] == 'P'
True
return
False
if board[player.pos] == 'V'
True
player.money += 100
False
if board[player.pos] == 'H'
True
house(player, houses, 20)
False
if board[player.pos] == 'M'
True
getMarr(player)
False
if board[player.pos] == 'C'
True
getPets(player)
False
if board[player.pos] == 'C'
True
getKids(player)
False
Page 5


Page 5
if board[player.pos] == 'N'
True
False
char pick
Ask user if they'd like to go to night school
if user inputs "y" or "Y"
False
tell user that they did not decide to go to night school
True
job(arr[i], careers, cc, jSize)
if board[player.pos] == 'R'
False
True
tell the player they are retired
return;
end of route function
start of winner function (Player *players, int pSize)
declare outside for loop
int most = -1, mostId
bool tie = false
declare inside for loop
i = 0
i++
i < pSize
False
players[i].money > most
False
most = players[i].money
mostId = i
tie = false;
players[i].money == most
tie = true;
if tie == true
True
output that there was a tie
False
players[mostId].winner = true
output winner name
end of winner fuinction
Page 6
end of results function
return;
display player[i] stats
i++
i < pSize
declare & initialize in for loop
int i = 0
if pick == 'y' or 'Y'
ask user if they would like to see results
cin >> pick
char pick
start of results function (Player *players, int pSize)


Page 6
start of spin function
return (rand() %10) + 1
end of spin function
start of move function (Player &plyr, int roll, int bSize)
plyr.pos += roll
if plyr.pos >= bSize - 2
False
True
plyr.pos = bSize -2
return
end of move function
start of stop check function
stopChk(Player &plyr, StpType types)
const int stops[] = {static_cast<int> (StpType::MARR),static_cast<int> (StpType::PETS),
static_cast<int>(StpType::KIDS), static_cast<int> (StpType::NIGHT)};
const int size = sizeof(stops) / sizeof(stops[0]);
plyr.stops < size && plyer.pos >= stops[player.stops]
True
False
plyr.pos = stops[plyr.stops];
plyr.stops++;
return
end of stop check function
start of action card function
action(Player &plyr, ActCard *actArr, int actSize)
int act = rand() %actSize
output actArr[act].event
plyer.aCount++ plyr.money += actArr[act].affect
if plyr.money < 0
True
plyr.money = 0
output players current money
return
end of action card function
start of pay function
(Player &plyr, Career *career)
plyr.money += career[plyr.career].income
output to player their money
end of pay function
start of pay check function
(Player &player, Career *careers, Career * cc, PayPos places)
const int payPos[] = {static_cast<int> (PayPos::PAY1), static_cast<int> (PayPos::PAY2), static_cast<int> (PayPos::PAY3),
static_cast<int> (PayPos::PAY4), static_cast<int> (PayPos::PAY5), static_cast<int> (PayPos::PAY6),
static_cast<int> (PayPos::PAY7), static_cast<int> (PayPos::PAY8), static_cast<int> (PayPos::PAY9),
static_cast<int> (PayPos::PAY10), static_cast<int> (PayPos::PAY11)};
const int size = sizeof(payPos) / sizeof(payPos[0]);
player.paid < size && player.pos >= payPos[player.paid]
False
True
if player.college = true
False
True
pay(player, careers)
pay(player, cc)
player.paid++
Page 7


Page 7
end of pay check function
start of get married function (Player &plyr)
tell player they landed on marriage stop
plyr.mrried = true
plyr.boat += 100
end of get married function
start of get kids function (Player &player)
cin.ignore(numeric_limits<streamsize>::max(), '\n');
int roll = spin()
roll <= 3
False
roll <= 6
True
plyr.kids = 1
False
roll <= 8
True
plyr.kids = 2
False
plyr.kids = 3
tell the player how many kids they get
end of get kids function
start of get pets function (Player &player)
cin.ignore(numeric_limits<streamsize>::max(), '\n');
int roll = spin()
roll <= 3
False
roll <= 6
True
plyr.pets= 1
False
roll <= 8
True
plyr.pets= 2
False
plyr.pets = 3
tell the player how many pets they get
end of get pets function
Page 8


start of retire function
(Player *players, House *houses, int pSize)
Page 8
Declare and initialize int i = 0 inside for loop
end of retire function
Page 9
False
i < pSize
i++
True
if players.[i].kids > 0
False
True
players[i].money += (players[i].kids *= 100)
if players.[i].pets > 0
False
True
players[i].money += (players[i].pets *= 100)
if players.[i].vCount > 0
False
True
players[i].money += (players[i].vCount *= 100)
if players.[i].aCount > 0
False
True
players[i].money += (players[i].aCount *= 100)
declare and initialize int j = 0 in for loop
j < 5
True
False
j++
players[i].hIds[j] != -1
True
False
int roll = spin()
cin.ignore(numeric_limits<streamsize>::max(), '\n');
roll % == 0
True
False
players[i].money += players[i].hOwned[j].sell2;
players[i].money += players[i].hOwned[j].sell1;


Page 9
start of job function
(Player &player, Career *carArr, Career *colArr, int jSize)
if player.college == true
jobSlct(player, arrArr, jSize);
False
True
jobSlct(player, colArr, jSize);
end of job function
start of job select function
(Player &player, Career *arr, int size)
Career choice1, choice2;
int id1 = rand() % size, id2 = rand() % size;
choice1 = arr[id1];
choice2 = arr[id2];
int pick;
prompt player to pick a job1 or job2
*use choice1 & choice 2 to display info.
pick != '1' && pick != '2'
True
False
cin.clear()
cin.ignore(numeric_limits<streamsize>::max(), '\n');
tell the user what job they picked or to input prompt correctly
cin >> pick
if pick == '1'
True
player.career = id1;
False
if pick == '2'
True
player.career = id2;
False
end of job select function
start of house function
(Player &plyr, House *arr, int size)
House choice1, choice2;
int pick, id1 = rand() % size, id2 = rand() % size;
id1 == id2
True
id2 = rand() % size
choice1 = arr[id1];
choice2 = arr[id2];
char pick
Page 10

Page 10
pick != '1' && pick != '2' && pick != '0'
True
pick = '/0'
prompt user to select house
cin >> pick
tell user to provide a valid input
pick == '1'
True
plyr.money < arr[id1].cost
True
tell user they're broke
False
declare & initialize int i = 0 inside for loop
i++
i < 5
True
if plyer.hIds[i] == -1
False
False
True
plyr.hIds[i] = id1;
plyr.hOwned[i] = choice1;
break;
False
False
pick == '0'
False
pick == '2'
True
True
tell player they chose not to buy a house
plyr.money < arr[id2].cost
False
declare & initialize int i = 0 inside for loop
i++
i < 5
True
if plyer.hIds[i] == -1
True
False
plyr.hIds[i] = id2;
plyr.hOwned[i] = choice2;
break;
False
True
tell user they're broke
end of house function
end of GameOfLifeV1.cpp